Controle d'un Jeu Video par Gestes de la Main

Vision par Ordinateur –

Cours d'Initiation

Master 2 Mathematiques, 2025–2026

Webcam $\longrightarrow$ MediaPipe $\longrightarrow$ Reseau de neurones $\longrightarrow$ WebSocket $\longrightarrow$ Jeu

Auteurs Christ Bohou
Djafarou Oulare

Depot `https://github.com/bohouchrist/projet_computer_vision`

Date Mars 2026

Resume. Ce projet consiste a controler le jeu Space Invaders par des gestes de la main captures en temps reel via une webcam. Trois approches ont ete experimentees : un CNN sur images brutes (Christ Bohou, echec a 34.3%), un MLP et un CNN sur un dataset Kaggle (Djafarou Oulare, quasi-100% en validation mais non generalisable en conditions reelles), et un reseau dense sur les coordonnees de la main extraites par MediaPipe (Christ Bohou, 99.6%, solution deployee).

Table des matières

1 Introduction et Objectif **3**
1.1 Gestes reconnus . . . 3
1.2 Vue d'ensemble du systeme deploye . . . 3
1.3 Synthese des trois approches . . . 3

2 Approche 1 : CNN sur Images Brutes **4**
2.1 Collecte des donnees . . . 4
2.2 Architecture du modele . . . 4
2.3 Resultats . . . 5

3 Approche 2 : MLP et CNN sur Images Kaggle **7**
3.1 Collecte des donnees . . . 7
3.2 Modele MLP . . . 7
3.3 Modele CNN . . . 8
3.4 Limite critique . . . 9

4 Approche 3 : Reseau Dense sur Landmarks MediaPipe **10**
4.1 MediaPipe HandLandmarker . . . 10
4.2 Collecte des donnees . . . 10
4.2.1 Normalisation geometrique . . . 10
4.3 Architecture du modele . . . 11
4.4 Entrainement . . . 11
4.5 Resultats . . . 12
4.6 Integration dans le jeu . . . 13

5 Comparaison et Discussion **14**
5.1 La representation avant le modele . . . 14
5.2 Limites . . . 14

6 Perspectives **14**

7 Conclusion **15**

A Structure du depot **16**

B Lancement **16**

1 Introduction et Objectif

L'objectif est de concevoir un **systeme complet de reconnaissance de gestes en temps reel** pour jouer a Space Invaders sans clavier ni souris. Ce projet couvre l'ensemble de la chaine de vision par ordinateur : collecte de donnees, entrainement de reseaux de neurones, et integration dans une application temps reel.

1.1 Gestes reconnus

Photo	Classe	Description	Action
	LEFT	Main droite, index pointe vers la gauche	Deplacer gauche
	RIGHT	Main gauche, index pointe vers la droite	Deplacer droite
	FIRE	Poing ferme, knuckles face camera	Tirer
	ENTER	Paume ouverte face camera	Demarrer
	NEUTRAL	Main au repos	Aucune action

1.2 Vue d'ensemble du systeme deploye

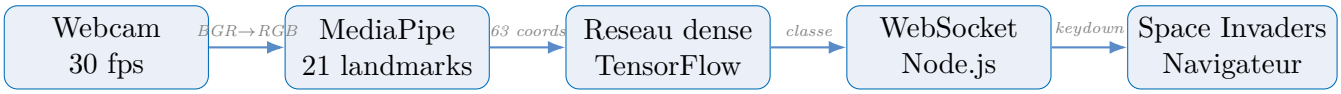

FIGURE 1 – Pipeline complet de controle gestuel en temps reel.

1.3 Synthese des trois approches

	Approche	Val. accuracy	Jouable
1	CNN sur images brutes – Christ Bohou	34.3%	Non
2	MLP + CNN Kaggle – Djafarou Oulare	∼100%	Non
3	**Reseau dense sur landmarks – Christ Bohou**	**99.6%**	**Oui**

2 Approche 1 : CNN sur Images Brutes

Approche 1 | Christ Bohou | ECHEC – val_accuracy 34.3%

Entrainement d'un CNN directement sur des images 64×64 pixels captures a la webcam. Le modele memorise le fond de la piece et ne generalise pas.

2.1 Collecte des donnees

Le script `1_collecter_images.py` capture automatiquement une image toutes les 300 ms. Chaque image est redimensionnee en 64×64 pixels RGB.

Listing 1 – Capture automatique d'images brutes.

```
IMG_SIZE   = 64
AUTO_DELAY = 0.3   # secondes entre deux captures

ret, frame = cap.read()
frame = cv2.flip(frame, 1)
img = cv2.resize(frame, (IMG_SIZE, IMG_SIZE))
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
cv2.imwrite(os.path.join(dossier, f"{classe}_{n:04d}.jpg"), img_rgb
    )
```

Classe	Images	Proportion
LEFT	502	19.9%
RIGHT	516	20.5%
FIRE	520	20.6%
ENTER	520	20.6%
NEUTRAL	463	18.4%
Total	**2 521**	100%

2.2 Architecture du modele

Listing 2 – Architecture CNN 3 blocs avec BatchNorm et Dropout.

```
modele = models.Sequential([
    # Bloc 1 - contours et textures simples
    layers.Conv2D(32,(3,3),activation='relu',padding='same',
                  input_shape=(64,64,3)),
    layers.BatchNormalization(),
    layers.Conv2D(32,(3,3),activation='relu',padding='same'),
    layers.MaxPooling2D(2,2),
    layers.Dropout(0.25),
    # Bloc 2 - formes complexes
```

```
    layers.Conv2D(64,(3,3),activation='relu',padding='same'),
    layers.BatchNormalization(),
    layers.Conv2D(64,(3,3),activation='relu',padding='same'),
    layers.MaxPooling2D(2,2),
    layers.Dropout(0.25),
    # Bloc 3 - configurations de main
    layers.Conv2D(128,(3,3),activation='relu',padding='same'),
    layers.BatchNormalization(),
    layers.MaxPooling2D(2,2),
    layers.Dropout(0.25),
    # Classification
    layers.Flatten(),
    layers.Dense(256, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.5),
    layers.Dense(5, activation='softmax'),   # 5 classes
])
modele.compile(optimizer='adam',
               loss='sparse_categorical_crossentropy',
               metrics=['accuracy'])
```

Parametres entraenables : 2 240 037.

Data augmentation : flip horizontal, rotation ±15%, zoom ±15%, luminosite ±20%, contraste ±20% – appliques en ligne sur chaque batch. EarlyStopping patience=8, ReduceLROnPlateau factor=0.5, split 80/20, batch 32, max 30 epochs.

2.3 Resultats

Echec – CNN images brutes

Meilleure val_accuracy : 34.3% a l'epoque 14 sur 22. Niveau chance aleatoire : 20% pour 5 classes.

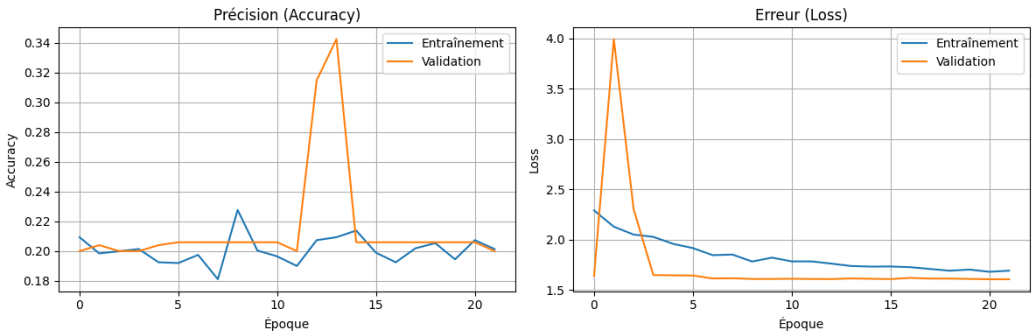

FIGURE 2 – Courbes d'entrainement – CNN images brutes. Precision de validation stagnant autour de 20% avec un seul pic isole a 34.3%. Aucune convergence stable.

Analyse. Le CNN apprend les correlations spurieuses entre le fond de la piece et la classe. Avec seulement 2 521 images dans un seul environnement, le modele memorise les couleurs du mur et les ombres plutot que la geometrie de la main. Pour generaliser, un CNN sur pixels bruts necessite des dizaines de milliers d'images dans des contextes varies.

3 Approche 2 : MLP et CNN sur Images Kaggle

Approche 2 | Djafarou Oulare | 100% en labo, inutilisable en reel

Dataset externe sur fond blanc, deux modeles testes. Performances quasi-parfaites en validation, echec complet en conditions reelles.

3.1 Collecte des donnees

Dataset *Hand Navigation Landmarks* (Kaggle) : images de mains sur fond blanc uniforme, 180×180 pixels, 4 classes.

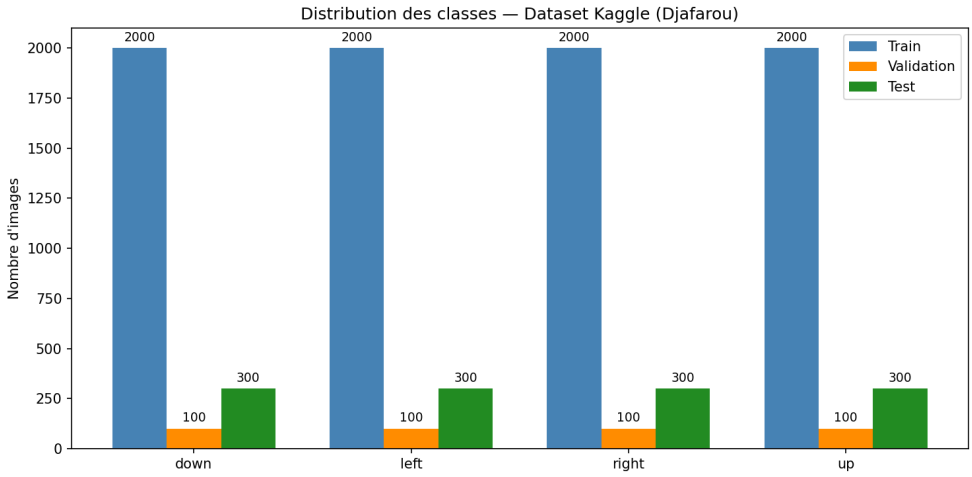

FIGURE 3 – Distribution du dataset Kaggle. 2 000 images par classe en train, 100 en validation, 300 en test.

Listing 3 – Chargement et pretraitement du dataset Kaggle.

```
train_ds = tf.keras.utils.image_dataset_from_directory(
    train_dir, class_names=['left','right','up','down'],
    image_size=(180,180), batch_size=32)

def normaliser(image, label):
    return tf.cast(image, tf.float32) / 255.0, label
def one_hot(image, label):
    return image, tf.one_hot(label, depth=4)

train_ds = train_ds.map(normaliser).map(one_hot)
```

3.2 Modele MLP

Listing 4 – Architecture MLP.

```
model = keras.Sequential([
    layers.Flatten(input_shape=(180,180,3)),    # 97 200 entrees
```

```
    layers.Dense(128, activation='relu'),
    layers.Dense(4, activation='softmax')
])
model.compile(optimizer='adam',
              loss='categorical_crossentropy', metrics=['accuracy']
                  )
model.fit(train_ds, validation_data=val_ds, epochs=100)
```

Parametres : 12 457 348.

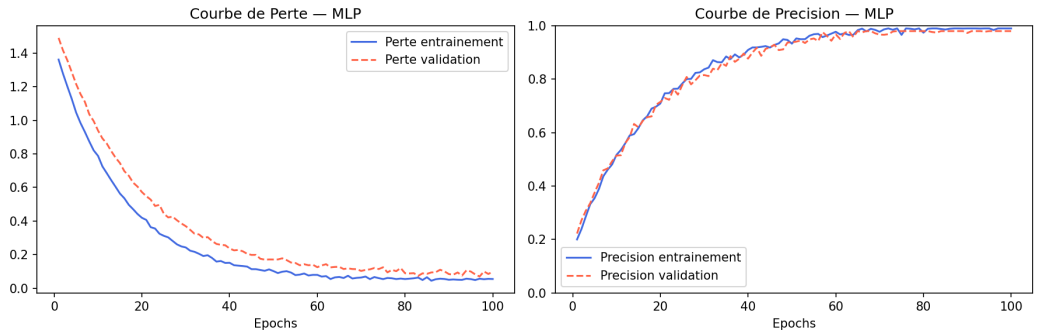

FIGURE 4 – Courbes MLP – convergence sur 100 epochs sur le dataset Kaggle.

3.3 Modele CNN

Listing 5 – Architecture CNN Kaggle.

```
model_cnn = Sequential([
    Conv2D(32,(3,3),activation='relu',input_shape=(180,180,3)),
    MaxPooling2D(2,2),
    Conv2D(64,(3,3),activation='relu'),
    MaxPooling2D(2,2),
    Flatten(),                # 118 336 valeurs
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(4, activation='softmax')
])
model_cnn.compile(optimizer='adam',
                  loss='categorical_crossentropy', metrics=['
                      accuracy'])
model_cnn.fit(train_ds, validation_data=val_ds, epochs=10)
```

Parametres : 15 167 044 dont 15 147 136 dans la seule couche Dense.

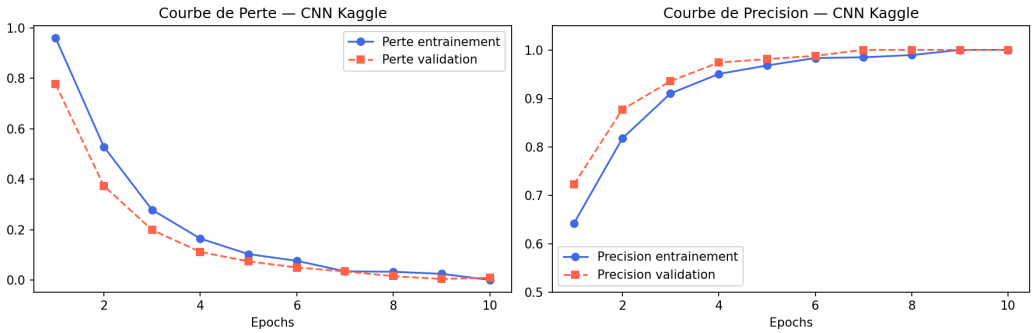

FIGURE 5 – Courbes CNN Kaggle – 100% en 10 epochs sur fond blanc.

3.4 Limite critique

Distribution shift – inutilisable en conditions reelles

Les deux modeles atteignent ∼100% sur Kaggle (fond blanc) mais **echouent sur une webcam reelle**.
Test concret : le CNN Kaggle predit `up` a 99.8% sur une image aleatoire de bruit pur – il a appris le fond blanc, pas les mains.

Ce phenomene est le **distribution shift** : l'ecart entre la distribution d'entrainement (fond blanc controlee) et la distribution reelle (fond de piece, eclairage variable) empeche toute generalisation.

4 Approche 3 : Reseau Dense sur Landmarks MediaPipe

Approche 3 | Christ Bohou | 99.6% – Solution deployee

Extraction de la geometrie de la main par MediaPipe puis classification du vecteur de coordonnees normalise. Invariant au fond, a l'eclairage et a la taille de la main.

4.1 MediaPipe HandLandmarker

Modele pre-entraine de Google (MobileNet). Detecte 21 points anatomiques (x, y, z) par image. En mode `VIDEO` : tracking leger entre frames, 30+ fps sur CPU.

Parametre	Valeur
min_hand_detection_confidence	0.6
min_hand_presence_confidence	0.5
min_tracking_confidence	0.5
Mode	VIDEO

4.2 Collecte des donnees

4.2.1 Normalisation geometrique

Pour chaque frame, le vecteur brut est normalise :

$$\tilde{\mathbf{p}}_i = \frac{\mathbf{p}_i - \mathbf{p}_0}{\max_j \|\mathbf{p}_j - \mathbf{p}_0\|_\infty} \tag{1}$$

$\mathbf{p}_0$ = poignet. Resultat : $\tilde{\mathbf{p}} \in [-1, 1]^{63}$, sans information de fond, couleur ou taille.

Listing 6 – Normalisation des landmarks.

```python
def normaliser(landmarks):
    pts = np.array([[lm.x, lm.y, lm.z] for lm in landmarks],
                   dtype=np.float32)
    pts -= pts[0]            # centrage sur le poignet
    s = np.max(np.abs(pts))
    if s > 0: pts /= s       # mise a l'echelle [-1, 1]
    return pts.flatten()     # vecteur de 63 valeurs
```

Cette normalisation est rigoureusement identique lors de la collecte, de l'entrainement et de l'inference.

Volume : environ 2 100 echantillons – 7 minutes de capture par classe.

4.3 Architecture du modele

Listing 7 – Reseau dense sur landmarks (4_entrainer_landmarks.py).

```
model = keras.Sequential([
    keras.Input(shape=(63,)),          # 21 points x 3 coordonnees

    keras.layers.Dense(128, activation='relu'),
    keras.layers.BatchNormalization(),
    keras.layers.Dropout(0.3),

    keras.layers.Dense(128, activation='relu'),
    keras.layers.BatchNormalization(),
    keras.layers.Dropout(0.3),

    keras.layers.Dense(64, activation='relu'),
    keras.layers.BatchNormalization(),
    keras.layers.Dropout(0.2),

    keras.layers.Dense(5, activation='softmax'),  # 5 classes
])
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy'],
)
```

Parametres : 33 925 – 66 fois moins que l'Approche 1.

Couche	Type	Sortie	Parametres
Input	–	63	0
Dense+BN+Dropout	ReLU	128	8 576
Dense+BN+Dropout	ReLU	128	16 896
Dense+BN+Dropout	ReLU	64	8 320
Dense	Softmax	5	325
Total			**33 925**

4.4 Entrainement

Listing 8 – Entrainement avec callbacks.

```
history = model.fit(
    X_train, y_train,
    epochs=50, batch_size=32,
    validation_data=(X_val, y_val),
    callbacks=[
```

```
        keras.callbacks.ModelCheckpoint(
            MODEL_PATH, monitor='val_accuracy', save_best_only=True
                ),
        keras.callbacks.EarlyStopping(
            monitor='val_accuracy', patience=15,
            restore_best_weights=True),
        keras.callbacks.ReduceLROnPlateau(
            monitor='val_loss', factor=0.5, patience=5),
    ]
)
```

Split 80/20 – 1 599 train / 501 validation. Convergence a l'epoque 13. Temps total : moins de 30 secondes sur CPU.

4.5 Resultats

Solution deployee – val_accuracy 99.6%

99.6% a l'epoque 13. 2 erreurs sur 501 echantillons. Entrainement en moins de 30 secondes sur CPU.

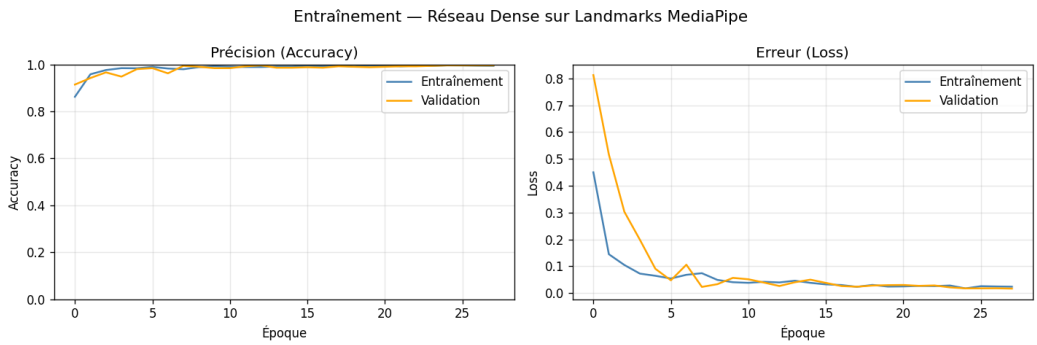

FIGURE 6 – Courbes – reseau dense sur landmarks. Convergence rapide et stable. Pas de sur-apprentissage.

Reel \ Predit	ENTER	FIRE	LEFT	NEUTRAL	RIGHT	Total
ENTER	**100**	0	0	0	0	100
FIRE	0	**100**	0	0	0	100
LEFT	0	0	**100**	0	0	100
NEUTRAL	0	1	1	**99**	0	101
RIGHT	0	0	0	0	**100**	100
Total	100	101	101	99	100	**501**

4.6 Integration dans le jeu

Listing 9 – Boucle d'inference temps reel.

```
SEUIL_CONFIANCE = 0.70

while True:
    ret, frame = cap.read()
    frame = cv2.flip(frame, 1)
    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    result = landmarker.detect_for_video(
        mp.Image(image_format=mp.ImageFormat.SRGB, data=rgb),
        timestamp_ms)
    if result.hand_landmarks:
        vecteur = normaliser(result.hand_landmarks[0]).reshape
            (1,-1)
        probs = model.predict(vecteur, verbose=0)[0]
        idx = np.argmax(probs)
        if probs[idx] >= SEUIL_CONFIANCE:
            geste = CLASSES[idx]
            if geste != 'NEUTRAL':
                commande_queue.put_nowait({'type': geste})
```

Architecture bi-thread : le thread principal gere la video et l'inference, un thread dedie gere l'envoi WebSocket sans bloquer la capture. Un hold timer de 180 ms dans Node.js maintient la touche enfoncee entre deux commandes consecutives.

5 Comparaison et Discussion

Critere	App. 1 CNN images	App. 2 Kaggle	App. 3 Landmarks
Val. accuracy	34.3%	∼100%	**99.6%**
Conditions reelles	Non	Non	**Oui**
Parametres	2.24 M	15.2 M	**33 925**
Entrainement CPU	5 min	15 min	**30 s**
Donnees	2 521 propres	8 000 externes	**2 100 propres**
Sensible au fond	Oui	Oui	**Non**
Dataset propre	Oui	Non	**Oui**

TABLE 1 – Comparaison des trois approches.

5.1 La representation avant le modele

App. 1 vs App. 3 dans les memes conditions :

	App. 1	App. 3
Dimension entree	12 288 pixels	63 coordonnees
Parametres	2 240 037	33 925
Val. accuracy	34.3%	**99.6%**

Reduire la dimension par 195 et les parametres par 66 a permis de passer de l'echec a la quasi-perfection. Le facteur determinant n'est pas la complexite du modele mais la **pertinence de la representation**.

5.2 Limites

FIRE moins robuste avec la main droite. Donnees FIRE collectees principalement avec la main gauche. La main droite produit un vecteur symetrique ($x_i \to -x_i$) que le modele ne reconnait pas toujours.

Impossibilite de tirer en se deplacant. Une seule commande par frame – LEFT et FIRE simultanement impossible.

6 Perspectives

Classes composites. Ajouter `LEFT_FIRE` et `RIGHT_FIRE` (poing incline) pour envoyer deux commandes simultanement.

FIRE deux mains. Recollecte avec les deux mains ou augmentation par symetrie ($x_i \to -x_i$).

LSTM temporel. Sequences de 5 a 10 frames pour gestes dynamiques et elimination des ambiguites statiques.

7 Conclusion

1. **La representation est decisive.** Pixels bruts $\to$ landmarks : dimension $\div 195$, parametres $\div 66$, accuracy 34% $\to$ 99.6%.
2. **Le score de validation ne garantit pas la generalisabilite.** 100% sur Kaggle, 0% en conditions reelles.
3. **L'ingenierie systeme compte autant que le modele.** Threading, hold timer, cooldowns sont indispensables pour une experience de jeu fluide.

A Structure du depot

```
projet_computer_vision/
|-- index.html / comment_jouer.html
|-- server.js / lancer_jeu.ps1
|-- hand_landmarker.task / rapport_projet.pdf
+-- cnn_training/
    |-- 1_collecter_images.py
    |-- 2_entrainer_cnn.py
    |-- 3_collecter_landmarks.py
    |-- 4_entrainer_landmarks.py
    |-- 5_jouer_avec_cnn.py
    |-- test_cnn_local.py
    +-- modeles/
        |-- gestes_landmarks.keras
        |-- classes_landmarks.txt
        +-- images_rapport/
```

B Lancement

Automatique : `.\lancer_jeu.ps1`

Manuel :

```
node server.js
python -m http.server 8000
cd cnn_training && python 5_jouer_avec_cnn.py
# http://localhost:8000
```

Test modele seul : `python test_cnn_local.py`